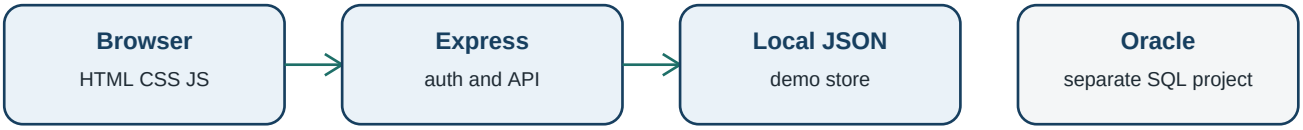
Vetwise Clinic Developer Handover

Architecture, file guide, code blocks, workflows, debugging, and maintenance

This is the developer companion to the project. It explains the purpose and interaction of every runtime file and the important blocks within each file. Read it beside the source when tracing an individual line.



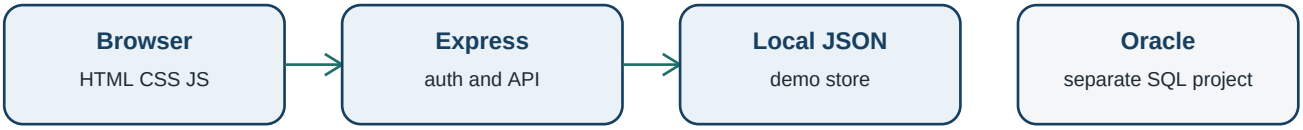
No live web-to-Oracle connection

The key fact is separation: the website uses a fictional local JSON store; the Oracle project is a separate SQL deliverable. No browser-to-Oracle connection exists.

1. Project map and system boundary

The root README is the starting map. sql contains the Oracle scripts; docs contains written artefacts and PDF builders; web contains the optional Express demonstration.

The web path is Browser -> Express -> local JSON. The database path is SQLcl -> Oracle schema. They model similar concepts but do not exchange live records.



No live web-to-Oracle connection

- AGENTS.md defines local rules: Oracle Free SQL only, fake data only, prescribed script names, documentation under docs, and no project-external changes.
- The root .gitignore and web/.gitignore keep local stores and .env files out of Git.

2. Configuration and data safety

web/.env.example names supported environment variables without providing credentials. package.json declares Express, dotenv, and node-oracledb for a future server-side integration. package-lock.json pins installed dependencies.

Ignored staff-accounts.json contains salts and password hashes. Ignored clinic-store.json contains demo mutations. They are private operational state, never source-controlled fixtures.

- Never commit passwords, invitation codes, Oracle connection strings, or local demo stores.
- Use a disposable local store in tests and delete it only when intentionally resetting the demo.

3. SQL run_all and safe teardown

sql/run_all.sql is the normal SQL entry point. Its WHENEVER SQLERROR directive exits on failure and rolls back the active transaction. SET DEFINE OFF stops substitution-variable surprises in sample literals.

The ordered @@ calls reflect dependencies. sql/00_drop_all.sql uses a small PL/SQL helper to drop views, package, triggers, then child tables before parent tables. Expected missing-object errors are ignored; unexpected failures are raised.

- Run from the project root with @sql/run_all.sql.
- Do not put connection details in the repository.

4. SQL DDL tables and constraints

sql/01_ddl.sql creates fourteen tables using Oracle NUMBER identities, VARCHAR2 text, and DATE values. Constraints are the first data-integrity boundary: keys, references, unique values, and checks are defined beside the affected columns.

Owners, species, breeds, veterinarians, services, medications, pets, and appointments describe clinic entities. Appointment services and prescriptions are detail/bridge tables. Vaccinations, invoices, payments, and appointment audit add clinical, finance, and history records.

- Snapshot prices on detail rows preserve historic charges when catalogue price changes.
- The invoice table stores status and date; totals are calculated from line items instead of copied.

5. SQL indexes and retained history

The DDL creates indexes for every foreign key and common search columns. This supports joins and protects parent rows from expensive child scans.

The function-based UQ_APPTS_SCHEDULED_VET_TIME index has values only for Scheduled rows. It prevents a veterinarian having two scheduled appointments at the exact same time while keeping completed and cancelled history valid.

Appointment audit retains INSERT and UPDATE events. Appointments are retained as history, so role grants do not include appointment DELETE.

6. SQL sample data and reports

sql/02_data.sql loads only fictional records. Single inserts are used for identity parent rows; PL/SQL loops build pets, vaccinations, appointments, invoice detail, invoices, and payments. The final COMMIT happens after a successful load.

sql/03_queries.sql has eighteen labelled report examples. Q5 aggregates each money source before joining, preventing service x prescription x payment multiplication. Q3 uses UNION ALL to show services and medications as independent item rows.

- Read each query from SELECT result, through joins, to filters, groups, and order.
- A report should calculate, not silently duplicate, monetary totals.

7. SQL views and date boundaries

sql/04_views.sql creates four shared reporting views: upcoming appointments, outstanding owner balances, veterinarian workload, and vaccinations due soon.

All 30-day logic uses a half-open range: >= today at midnight and < midnight after day 30. This includes every valid time on day 30. The same rule is used in report queries and package reminders.

- A view is a named query, not copied data.
- Balance reporting excludes VOID and non-positive balances.

8. Package contract and booking block

sql/05_package.sql starts with the public package specification. The body implements booking, invoice calculation, vaccination registration, and reminders. Keep external callers dependent on the specification, not body variables.

book_appointment validates IDs, date, reason, pet existence, active veterinarian, and scheduled exact-time conflict before inserting. The application-error codes make invalid input predictable and testable.

- Read package code in this order: specification, declarations, validation, queries, mutation, exception block.
- USER_ERRORS should be empty after compile.

9. Package billing and reminders block

calculate_invoice_total resolves the invoice appointment, sums service and prescription quantity times snapshot price independently, then returns the combined result. NVL ensures an empty line type contributes zero.

register_vaccination validates a real pet, nonblank name, and non-future date, then adds a 365-day due date. vaccination_reminders validates a whole day range, opens a parameterized cursor, prints results, and closes the cursor even on error.

10. Trigger blocks: audit and inventory

sql/06_triggers.sql has an appointment audit trigger using INSERTING, OLD, NEW, USER, and SYSDATE. It records new appointments and changes to selected business fields.

The compound prescription trigger adjusts inventory for insert, quantity change, medicine change, and delete. It reserves replacement stock before releasing old stock and rejects insufficient stock with clear application errors.

Paid-service-line and paid-prescription triggers reject changes after the related invoice has payments, preserving financial history.

11. Trigger blocks: invoice state

The invoice-date trigger supplies SYSDATE only when an insert gives no date. The invoice-status validation trigger checks a non-void invoice against detail total and existing payments; it rejects overpayment and mismatched manual status.

The payment compound trigger remembers affected invoice IDs per row, then after the statement recalculates totals once per invoice. It derives UNPAID, PARTIAL, or PAID status and refuses payments for a void invoice.

- Use after-statement work for a multi-row payment change.
- The script’s demonstration blocks use savepoints and rollback.

12. SQL roles and regression tests

sql/07_security.sql checks SESSION_PRIVS before role creation. It avoids aborting a normal schema owner that lacks CREATE ROLE and handles Oracle’s VET user/role name collision honestly.

sql/08_tests.sql contains 22 numbered tests. Together with trigger demonstrations it produces 34 PASS messages in a successful full build. Tests cover constraints, package errors, audit, inventory lifecycle, payments, paid-line protection, date range inclusion, and retained history.

- Every temporary SQL test change is rolled back.
- Run the rebuild in a disposable schema during maintenance.

13. Express server entry file

web/server.js imports Express and path, loads environment values, builds the app, enables JSON parsing, exposes static public files, and mounts auth before the protected API router.

The unknown /api handler returns JSON 404 before the HTML catch-all. The four-argument final middleware handles malformed JSON and unexpected errors. The require.main guard starts a listener only when launched directly, allowing tests to import the app.

14. auth.js accounts and roster linking

web/routes/auth.js imports crypto, file utilities, Express, and clinic persistence. Module constants name store paths, invitation-code settings, cookie lifetime, and scrypt settings.

ensureVeterinarianProfile links a veterinarian account to one roster profile by account ID, email, or controlled legacy-name matching. getVeterinarianRoster returns only account-backed doctors, so booking never presents a doctor who cannot sign in.

- persistAccounts writes temporary private JSON then renames it atomically.
- publicAccount deliberately omits salt and password hash.

15. auth.js sessions, validation, and routes

setSession creates a random token, stores account ID and expiry in memory, and sends an HttpOnly SameSite cookie. getSessionAccount expires stale tokens. Rate limiting records attempts per IP and route during a 15-minute window.

Registration validates all inputs, verifies role-specific invitation codes, derives a salted scrypt hash, persists account and any new roster profile, and returns only public user data. Login hashes unknown users with a dummy salt to make the slow path similar.

- requireAuth attaches a public account to the request.
- requireRole is the reusable authorization guard for protected route actions.

16. api.js shared helpers and dashboard

web/routes/api.js mounts requireAuth once for all following routes. Constants describe ISO shape, time shape, duration, active statuses, and field length limits.

isRealDate detects rollover values; isRealTime checks clock range and five-minute slots; isWithinClinicHours enforces Monday-Saturday and a full 30-minute fit. The dashboard scopes schedule data by role and roster before calculating metrics.

17. api.js mutations and transitions

Patient creation is admin-only, validates fields, appends in memory, saves, and removes the new row if saving fails. Appointment creation resolves a patient and linked doctor, validates date/time/reason, and rejects active 30-minute overlap for either doctor or patient.

Appointment PATCH enforces the state machine: only the assigned veterinarian may confirm pending work; an admin may cancel active work or complete eligible confirmed or legacy scheduled work after its time. Invoice PATCH is an intentionally simple admin-only mark-paid demo action.

18. clinic.js demo data and atomic store

web/data/clinic.js exports local-date helpers, clinic facts, seeded arrays, and persistence. Local date components avoid UTC date shifts. Initial records use relative dates so the demo remains useful.

When a store exists, collection contents are replaced in place so imported modules retain stable array references. persistClinicData writes a random temporary file, renames it, then applies restrictive permission mode.

- This is not concurrent database storage.
- A future Oracle integration belongs on the server and requires pooling, parameterized SQL, and mapping design.

19. index.html markup contract

web/public/index.html is the static application shell. It defines auth forms, sidebar, dashboard, tables, filters, dialogs, toast, and error regions. main.js depends on its IDs and classes.

Use semantic form controls, native dialog elements, labels, and live text feedback. Role-specific elements start hidden but server-side checks still decide whether an action is allowed.

20. main.js state and rendering blocks

web/public/js/main.js begins with state and a request wrapper. request parses JSON, detects a protected-route 401, clears local user state, and creates a consistent error object. JSON helpers centralize POST and PATCH setup.

escapeHtml protects values inserted into template strings. renderDashboard, renderPatients, renderAppointments, and renderInvoices transform loaded state into DOM content. Each renderer shows only role-appropriate actions and an empty state when needed.

21. main.js forms, refresh, and events

Dialog helpers reset fields, populate select choices from state, set date/time constraints, and open native dialogs. Submit handlers POST data, refresh state after success, and render server validation errors without assuming a successful request.

loadClinicData uses Promise.all for independent reads. refreshData repeats it after mutations. A delegated click listener handles buttons created by later renders; direct listeners handle fixed forms, filters, search, logout, and dialog dismissals.

22. styles.css visual and access blocks

web/public/styles.css is organised from theme variables and base rules through shell layout, reusable components, tables/dialogs, and responsive media queries. Read top-down to find a selector’s inherited context.

The sidebar flex and sticky rules keep navigation usable. Content min-width prevents wide tables forcing page overflow. Table wrappers handle horizontal scrolling. Reduced-motion and transparency fallbacks preserve readable use on varied devices.

- ARIA states, focus placement, and labels support keyboard and assistive technology.
- CSS is presentation; it is not the authorization boundary.

23. Tests and DOM checker

web/test/api.test.js starts the imported Express app on an ephemeral port with temporary local stores. It registers accounts, keeps cookie state, and verifies static delivery, authentication, role scope, booking, validation, transitions, invoices, persistence, unknown routes, and malformed JSON.

web/check_dom.js compares selector targets used by main.js with index.html. It catches missing IDs or classes after a markup edit. Run npm test and node check_dom.js from web before committing.

24. Documentation and PDF files

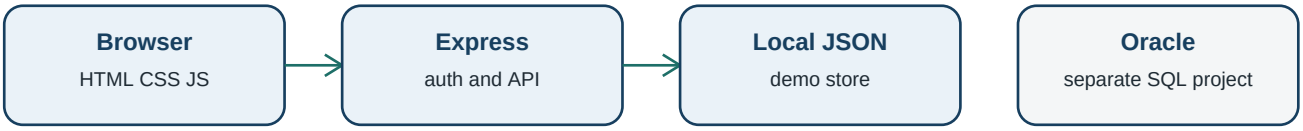
docs/requirements.md is the project scope. design.md, erd.md, and normalization.md describe the relational model. decisions.md records intentional choices. checklist.md is completion evidence. backup.md describes safe database operations.

docs/user-guide.md and docs/technical-handover.md are editable prose sources. docs/build_pdf_guides.js is the original plain-text PDF builder. docs/create_guides_pdf.py creates the current styled PDFs with page headers, tables, and a boundary diagram.

25. End-to-end workflow and line tracing

Appointment flow: administrator browser -> POST API -> validation -> in-memory object -> atomic JSON write -> JSON response -> refresh. Confirmation flow: assigned veterinarian -> PATCH -> session and assignment checks -> save -> refreshed worklist. Completion/cancellation follows the admin state rules.

For Oracle: package or direct SQL writes -> constraints and triggers -> stored rows -> report/view queries. Detail lines determine invoice total; payment trigger determines status; paid-line triggers preserve already-paid totals; stock and audit triggers protect operational history.



No live web-to-Oracle connection

26. Developer runbook and final checklist

For JavaScript changes, run node --check on changed files, npm test, and node check_dom.js. For SQL changes, run @sql/run_all.sql in a disposable Oracle schema and confirm no USER_ERRORS, no FAIL messages, and 34 PASS messages.

For any line you change, trace input, validation, state mutation, persistence, response, caller, and test. Then update the appropriate guide and decisions record. This is the practical way to explain every important line and block accurately.

- Inspect git status and git diff --check.
- Never stage ignored stores or secrets.
- Regenerate and visually inspect PDFs after documentation changes.
- Commit source and documentation, then push the intended branch.